Prometheus Query Workshop

2026-04-21

Introduction

This workshop is designed to help you become proficient with Prometheus Query Language (PromQL). You will learn how to write effective queries to extract meaningful insights from your metrics data, from basic selectors to advanced aggregation patterns.

Workshop Structure

This workshop consists of four modules:

- **Module 1: Prometheus Query Language Fundamentals** - Learn the basics of PromQL, including metric names, labels, data types, and basic query syntax
- **Module 2: Selectors and Filtering** - Master label selectors, regular expressions, and filtering techniques (includes exercises)
- **Module 3: Aggregation Functions** - Understand how to use aggregation functions like sum, average, quantiles, and top-k (includes exercises)
- **Module 4: Advanced Query Patterns** - Explore rate functions, subqueries, logical operators, and optimization techniques (includes exercises)

Prerequisites

This workshop assumes the following:

- Basic understanding of Prometheus concepts (metrics, labels, time series)
- Access to an OpenShift 4.16 cluster with Prometheus installed
- Ability to access the Prometheus web UI or use the Prometheus API
- Familiarity with basic command-line operations

If you are new to Prometheus, we recommend reviewing the Prometheus documentation before proceeding.

Expectations

After completing this workshop, you will be able to:

- Write basic PromQL queries using selectors and filters
- Apply aggregation functions to summarize metrics across multiple time series
- Construct advanced queries using rate functions, subqueries, and logical operators
- Debug and optimize Prometheus queries for better performance
- Use PromQL effectively in Grafana dashboards and alerting rules

Module 1: Prometheus Query Language Fundamentals

Content

This module introduces you to the fundamentals of Prometheus Query Language (PromQL). You will learn the basic concepts, data types, and syntax needed to write effective Prometheus queries.

Understanding Prometheus Metrics

Prometheus stores metrics as time series data. Each time series is identified by a metric name and a set of key-value pairs called labels.

Metric Names

Metric names describe what is being measured. They must follow these rules:

- Contain only letters, digits, underscores, and colons
- Start with a letter or underscore
- Cannot be reserved keywords

Examples of valid metric names:

- `http_requests_total`
- `node_cpu_seconds_total`
- `container_memory_usage_bytes`

Labels

Labels are key-value pairs that provide additional dimensions to your metrics. They allow you to filter and aggregate data.

Common label examples:

- `instance` - The target being monitored
- `job` - The job name that collected the metric
- `namespace` - Kubernetes namespace (in OpenShift)
- `pod` - Kubernetes pod name
- `container` - Container name

Basic Query Syntax

A PromQL query consists of a metric name, optionally followed by label selectors in curly braces. '{}'

Simple Metric Selection

To select all time series for a metric:

```
http_requests_total
```

This returns all time series with the metric name `http_requests_total`.

Label Selectors

Label selectors filter time series by matching label values. Use curly braces `{}` to specify label filters.

Select all HTTP requests for a specific instance:

```
http_requests_total{instance="localhost:9090"}
```

Label Matching Operators

PromQL supports several matching operators:

- `=` - Exact match
- `!=` - Not equal
- `=~` - Regular expression match
- `!~` - Regular expression not match

Examples:

```
# Exact match
http_requests_total{method="GET"}

# Not equal
http_requests_total{method!="GET"}

# Regular expression match
http_requests_total{method=~"GET|POST"}

# Regular expression not match
http_requests_total{method!~"GET|POST"}
```

Data Types

PromQL has four main data types:

Instant Vector

An instant vector is a set of time series, each containing a single sample at a given timestamp. This

is the most common result type.

Example:

```
http_requests_total{method="GET"}
```

Range Vector

A range vector is a set of time series, each containing a range of data points over time. Range vectors are used with functions like `rate()`. '[]' is used to specify the time range.

Example:

```
http_requests_total{method="GET"}[5m]
```

Scalar

A scalar is a simple numeric floating-point value.

String

A string is a simple string value (used primarily for labels).

Time Ranges

When working with range vectors, you specify a time range using duration syntax:

- `s` - seconds
- `m` - minutes
- `h` - hours
- `d` - days
- `w` - weeks
- `y` - years

Examples:

```
# Last 5 minutes
http_requests_total[5m]

# Last 1 hour
http_requests_total[1h]

# Last 2 days
http_requests_total[2d]
```

Accessing Prometheus in OpenShift

In OpenShift 4.16, you can access Prometheus through the web console or via the API.

Accessing the Prometheus Web UI

1. Log in to your OpenShift cluster web console
2. Navigate to the `openshift-monitoring` namespace
3. Find the Prometheus service or route
4. Access the Prometheus UI

Alternatively, you can port-forward to the Prometheus service:

```
oc port-forward -n openshift-monitoring svc/prometheus-k8s 9090:9090
```

Then access Prometheus at `http://localhost:9090`.

Using the Prometheus API

You can query Prometheus programmatically using its HTTP API:

```
# Get an authentication token
TOKEN=$(oc whoami -t)

# Query Prometheus API
curl -H "Authorization: Bearer $TOKEN" \
  "https://prometheus-k8s-openshift-monitoring.apps.<cluster-
domain>/api/v1/query?query=http_requests_total"
```

Common Built-in Metrics in OpenShift

OpenShift 4.16 exposes many useful metrics. Here are some common ones:

- `container_cpu_usage_seconds_total` - CPU usage per container
- `container_memory_usage_bytes` - Memory usage per container
- `kube_pod_info` - Pod information
- `kube_node_info` - Node information
- `openshift_build_info` - Build information
- `cluster:capacity_cpu_cores:sum` - Total CPU capacity

Summary

In this module, you learned:

- The structure of Prometheus metrics (metric names and labels)

- Basic PromQL query syntax
- Label selectors and matching operators
- PromQL data types (instant vector, range vector, scalar, string)
- How to specify time ranges
- How to access Prometheus in OpenShift 4.16

In the next module, you will learn more advanced selector techniques and filtering patterns.

Module 2: Selectors and Filtering

Content

This module covers advanced selector techniques and filtering patterns in PromQL. You will learn how to combine multiple label selectors, use regular expressions effectively, and filter time series based on label values.

Combining Multiple Label Selectors

You can combine multiple label selectors in a single query. All selectors must match for a time series to be included.

AND Logic (Default)

When you specify multiple label selectors, they are combined with AND logic:

```
http_requests_total{method="GET", status="200"}
```

This query returns time series where both `method="GET"` AND `status="200"`.

Complex Selector Examples

```
# Multiple exact matches
container_memory_usage_bytes{namespace="default", pod=~"web-.*"}

# Mix of exact and regex matches
kube_pod_info{namespace="production", node!~"worker-0.*"}
```

Regular Expression Patterns

Regular expressions are powerful tools for matching patterns in label values.

Basic Regex Patterns

```
# Match any pod starting with "web-"
kube_pod_info{pod=~"web-.*"}

# Match pods ending with "-prod"
kube_pod_info{pod=~".*-prod"}

# Match pods containing "api"
kube_pod_info{pod=~".*api.*"}
```

Anchored Regex Patterns

Use `^` for start anchor and `$` for end anchor:

```
# Must start with "web-"
kube_pod_info{pod=~"^web-.*"}

# Must end with "-v1"
kube_pod_info{pod=~".*-v1$"}

# Exact match using anchors
kube_pod_info{pod=~"^web-server-v1$"}
```

Character Classes

```
# Match any digit
kube_pod_info{pod=~".*-v[0-9]+"}

# Match specific characters
kube_pod_info{namespace=~"^(dev|staging|prod)$"}
```

Filtering by Label Existence

You can check if a label exists or doesn't exist using the `label_replace` function or by using negative regex patterns.

Checking for Label Presence

While PromQL doesn't have a direct "label exists" operator, you can use workarounds:

```
# This will only match if the label exists and has a value
# (empty string values won't match)
container_memory_usage_bytes{container!=""}
```

Negative Filtering

Use the `!=` and `!~` operators to exclude specific values.

Excluding Specific Values

```
# Exclude a specific namespace
container_cpu_usage_seconds_total{namespace!="kube-system"}

# Exclude multiple values using regex
container_cpu_usage_seconds_total{namespace!~"kube-system|openshift-.*"}
```

Working with OpenShift-Specific Labels

OpenShift 4.16 adds several useful labels to metrics. Understanding these helps you write effective queries.

Common OpenShift Labels

- `namespace` - The Kubernetes namespace
- `pod` - The pod name
- `container` - The container name
- `node` - The node name
- `deployment` - The deployment name
- `replicaset` - The replica set name
- `service` - The service name

Filtering by Namespace

```
# All metrics in a specific namespace
container_memory_usage_bytes{namespace="my-app"}

# All metrics except system namespaces
container_memory_usage_bytes{namespace!~"kube-.*|openshift-.*"}
```

Filtering by Pod Patterns

```
# All pods for a specific deployment
kube_pod_info{deployment="web-server", namespace="production"}

# Pods matching a naming pattern
container_cpu_usage_seconds_total{pod=~"web-server-.*"}
```

Combining Queries with Operators

You can combine multiple queries using arithmetic and comparison operators.

Arithmetic Operators

```
# Add two metrics
container_memory_usage_bytes{namespace="app1"} +
container_memory_usage_bytes{namespace="app2"}

# Multiply by a scalar
container_memory_usage_bytes{namespace="my-app"} * 1024
```

Comparison Operators

Comparison operators return 1 for true, 0 for false:

```
# Find containers using more than 1GB of memory
container_memory_usage_bytes > 1073741824

# Find containers using less than 100MB
container_memory_usage_bytes < 104857600
```

Best Practices for Selectors

Use Specific Selectors

Be as specific as possible to reduce query execution time:

```
# Good: Specific namespace and pod pattern
container_memory_usage_bytes{namespace="production", pod=~"web-.*"}

# Less efficient: Too broad
container_memory_usage_bytes
```

Optimize Regex Patterns

Simple regex patterns are faster than complex ones:

```
# Good: Simple prefix match
pod=~"web-.*"

# Less efficient: Complex pattern
pod=~".*web.*server.*v[0-9]+.*"
```

Avoid Overly Broad Negations

Negative selectors can be expensive. Use positive matches when possible:

```
# Better: Positive match
container_memory_usage_bytes{namespace="my-app"}

# Less efficient: Negative match (if you have many namespaces)
container_memory_usage_bytes{namespace!="kube-system"}
```

Summary

In this module, you learned:

- How to combine multiple label selectors
- Regular expression patterns and their usage
- Filtering by label existence and exclusion
- OpenShift-specific labels and how to use them
- Combining queries with operators
- Best practices for writing efficient selectors

In the next module, you will learn about aggregation functions, which allow you to summarize data across multiple time series.

Exercises

Exercise 1: Basic Label Filtering

In this exercise, you will practice writing queries with label selectors to filter metrics in your OpenShift cluster.

Step 1: Access Prometheus

1. Open a terminal and authenticate to your OpenShift cluster:

```
oc login <your-cluster-url>
```

1. Port-forward to the Prometheus service:

```
oc port-forward -n openshift-monitoring svc/prometheus-k8s 9090:9090
```

1. Open your browser and navigate to `http://localhost:9090`

Step 2: Query Container Memory Usage

Write a query to find all container memory usage metrics in the `default` namespace:

```
container_memory_usage_bytes{namespace="default"}
```

Execute this query in the Prometheus UI and observe the results.

Step 3: Filter by Pod Pattern

Modify the query to only show metrics for pods whose names start with "web-":

```
container_memory_usage_bytes{namespace="default", pod=~"^web-.*"}
```

Step 4: Exclude System Namespaces

Write a query to show container CPU usage for all namespaces except system namespaces:

```
container_cpu_usage_seconds_total{namespace!~"kube-.*|openshift-.*"}
```

Exercise 2: Regular Expression Patterns

Practice using regular expressions to match patterns in label values.

Step 1: Match Pod Names with Version Numbers

Write a query to find all pods with names containing a version number pattern (e.g., "v1", "v2"):

```
kube_pod_info{pod=~".*-v[0-9]+.*"}
```

Step 2: Match Specific Deployment Pattern

Find all pods that belong to deployments with names matching the pattern "app-*-prod":

```
kube_pod_info{deployment=~"^app-.*-prod$"}
```

Step 3: Exclude Test and Dev Environments

Write a query to show container memory usage excluding test and development namespaces:

```
container_memory_usage_bytes{namespace!~".*-test|.*-dev"}
```

Exercise 3: Combining Multiple Filters

Practice combining multiple label selectors and operators.

Step 1: Filter by Multiple Conditions

Write a query to find container memory usage in the "production" namespace for containers with names starting with "app-":

```
container_memory_usage_bytes{namespace="production", container=~"^app-.*"}
```

Step 2: Use Comparison Operators

Find all containers using more than 500MB of memory:

```
container_memory_usage_bytes > 524288000
```

Step 3: Combine Filters and Comparisons

Find containers in the "production" namespace using more than 1GB of memory:

```
container_memory_usage_bytes{namespace="production"} > 1073741824
```

Exercise 4: OpenShift-Specific Queries

Practice writing queries using OpenShift-specific labels and metrics.

Step 1: Query Pod Information

List all pods in a specific namespace with their node assignments:

```
kube_pod_info{namespace="your-namespace"}
```

Replace "your-namespace" with an actual namespace in your cluster.

Step 2: Filter by Node

Find all pods running on a specific node:

```
kube_pod_info{node="<node-name>"}
```

Replace `<node-name>` with an actual node name from your cluster. You can find node names using:

```
oc get nodes
```

Step 3: Query Build Information

Query OpenShift build information for builds in a specific namespace:

```
openshift_build_info{namespace="your-namespace"}
```

Verification

After completing these exercises, verify your understanding:

- Can you write a query that filters metrics by multiple labels?
- Can you use regular expressions to match patterns?

- Can you exclude specific values using negative operators?
- Can you combine filters with comparison operators?

If you can answer yes to all these questions, you are ready to proceed to the next module on aggregation functions.

Module 3: Aggregation Functions

Content

This module covers PromQL aggregation functions, which allow you to summarize data across multiple time series. You will learn how to group metrics, calculate sums, averages, and other statistical functions.

Understanding Aggregation

Aggregation functions reduce multiple time series into a single time series by grouping them and applying a function (sum, average, max, etc.) to each group.

Basic Aggregation Syntax

```
<aggregation-function>([parameter,] <vector expression>) [without|by (<label list>)]
```

The `by` Clause

The `by` clause specifies which labels to keep when grouping:

```
sum(container_memory_usage_bytes) by (namespace)
```

This groups all time series by namespace and sums the memory usage for each namespace.

The `without` Clause

The `without` clause specifies which labels to remove before grouping:

```
sum(container_memory_usage_bytes) without (pod, container)
```

This removes the `pod` and `container` labels, then sums the remaining groups.

Sum Aggregation

The `sum()` function adds up all values in each group.

Basic Sum

```
# Total memory usage across all containers
sum(container_memory_usage_bytes)
```

Sum by Label

```
# Total memory usage per namespace
sum(container_memory_usage_bytes) by (namespace)

# Total CPU usage per node
sum(container_cpu_usage_seconds_total) by (node)
```

Sum with Multiple Labels

```
# Total memory usage per namespace and pod
sum(container_memory_usage_bytes) by (namespace, pod)
```

Average Aggregation

The `avg()` function calculates the average value for each group.

Basic Average

```
# Average memory usage across all containers
avg(container_memory_usage_bytes)
```

Average by Label

```
# Average memory usage per namespace
avg(container_memory_usage_bytes) by (namespace)

# Average CPU usage per node
avg(container_cpu_usage_seconds_total) by (node)
```

Min and Max Aggregations

The `min()` and `max()` functions find the minimum and maximum values in each group.

Minimum Values

```
# Minimum memory usage per namespace
min(container_memory_usage_bytes) by (namespace)
```

Maximum Values

```
# Maximum memory usage per namespace
max(container_memory_usage_bytes) by (namespace)
```

Count Aggregation

The `count()` function counts the number of time series in each group.

Count Time Series

```
# Count of containers per namespace
count(container_memory_usage_bytes) by (namespace)

# Count of pods per node
count(kube_pod_info) by (node)
```

Statistical Aggregations

PromQL provides several statistical aggregation functions.

Standard Deviation

The `stddev()` function calculates the standard deviation:

```
# Standard deviation of memory usage per namespace
stddev(container_memory_usage_bytes) by (namespace)
```

Variance

The `stdvar()` function calculates the variance:

```
# Variance of memory usage per namespace
stdvar(container_memory_usage_bytes) by (namespace)
```

Quantiles

The `quantile()` function calculates quantiles over grouped time series:

```
# 95th percentile of memory usage per namespace
quantile(0.95, container_memory_usage_bytes) by (namespace)

# 50th percentile (median) of CPU usage per node
quantile(0.50, container_cpu_usage_seconds_total) by (node)
```

Common quantile values:
* `0.50` - Median
* `0.90` - 90th percentile
* `0.95` - 95th percentile
* `0.99` - 99th percentile

Top-K and Bottom-K

The `topk()` and `bottomk()` functions return the top or bottom K time series by value.

Top-K

```
# Top 5 containers by memory usage
topk(5, container_memory_usage_bytes)

# Top 3 namespaces by total memory usage
topk(3, sum(container_memory_usage_bytes) by (namespace))
```

Bottom-K

```
# Bottom 5 containers by memory usage
bottomk(5, container_memory_usage_bytes)

# Bottom 3 nodes by CPU usage
bottomk(3, sum(container_cpu_usage_seconds_total) by (node))
```

Combining Aggregations

You can combine multiple aggregation functions to create complex queries.

Nested Aggregations

```
# Average of maximum memory usage per namespace
avg(max(container_memory_usage_bytes) by (namespace, pod)) by (namespace)
```

Aggregation with Filtering

```
# Sum of memory usage for production namespaces only
sum(container_memory_usage_bytes{namespace=~"prod.*"}) by (namespace)
```

Rate Functions with Aggregation

Rate functions are often used with aggregations to calculate rates across multiple time series.

Rate and Sum

```
# Total request rate per service
sum(rate(http_requests_total[5m])) by (service)
```

Rate and Average

```
# Average request rate per namespace
avg(rate(http_requests_total[5m])) by (namespace)
```

Common Aggregation Patterns in OpenShift

Total Resource Usage per Namespace

```
# Total CPU usage per namespace
sum(container_cpu_usage_seconds_total) by (namespace)

# Total memory usage per namespace
sum(container_memory_usage_bytes) by (namespace)
```

Average Resource Usage per Node

```
# Average CPU usage per node
avg(container_cpu_usage_seconds_total) by (node)

# Average memory usage per node
avg(container_memory_usage_bytes) by (node)
```

Pod Count per Namespace

```
# Number of pods per namespace
count(kube_pod_info) by (namespace)
```

Top Resource Consumers

```
# Top 10 pods by memory usage
topk(10, container_memory_usage_bytes)

# Top 5 namespaces by total CPU usage
topk(5, sum(container_cpu_usage_seconds_total) by (namespace))
```

Best Practices

Choose Appropriate Aggregation

- Use `sum()` for additive metrics (counters, bytes)
- Use `avg()` for metrics where you want the average
- Use `max()` or `min()` when you need extremes

- Use `quantile()` for percentile analysis

Group by Meaningful Labels

Group by labels that make sense for your use case:

```
# Good: Group by namespace for resource planning
sum(container_memory_usage_bytes) by (namespace)

# Less useful: Group by instance (too granular)
sum(container_memory_usage_bytes) by (instance)
```

Avoid Over-Aggregation

Don't aggregate away information you might need:

```
# Too aggregated: Loses pod-level detail
sum(container_memory_usage_bytes)

# Better: Keeps namespace detail
sum(container_memory_usage_bytes) by (namespace)
```

Summary

In this module, you learned:

- How aggregation functions work in PromQL
- Using `by` and `without` clauses for grouping
- Common aggregation functions: `sum()`, `avg()`, `min()`, `max()`, `count()`
- Statistical functions: `stddev()`, `stdvar()`, `quantile()`
- Top-K and bottom-K functions
- Combining aggregations with rate functions
- Common aggregation patterns for OpenShift
- Best practices for effective aggregation

In the next module, you will learn advanced query patterns including rate functions, subqueries, and logical operators.

Exercises

Exercise 1: Basic Aggregations

Practice using basic aggregation functions to summarize metrics.

Step 1: Sum Aggregation

Write a query to calculate the total memory usage across all containers in your cluster:

```
sum(container_memory_usage_bytes)
```

Execute this query in Prometheus and observe the result.

Step 2: Sum by Namespace

Modify the query to show total memory usage per namespace:

```
sum(container_memory_usage_bytes) by (namespace)
```

Compare the results with the previous query. How many namespaces do you have?

Step 3: Average Memory Usage

Write a query to calculate the average memory usage per namespace:

```
avg(container_memory_usage_bytes) by (namespace)
```

Compare the average with the sum. What insights can you draw?

Exercise 2: Min, Max, and Count

Practice using min, max, and count aggregation functions.

Step 1: Maximum Memory Usage

Find the maximum memory usage per namespace:

```
max(container_memory_usage_bytes) by (namespace)
```

Which namespace has the highest maximum memory usage?

Step 2: Minimum Memory Usage

Find the minimum memory usage per namespace:

```
min(container_memory_usage_bytes) by (namespace)
```

Step 3: Count Containers

Count the number of containers per namespace:

```
count(container_memory_usage_bytes) by (namespace)
```

Which namespace has the most containers?

Exercise 3: Statistical Aggregations

Practice using statistical functions like quantiles and standard deviation.

Step 1: Median Memory Usage

Calculate the median (50th percentile) memory usage per namespace:

```
quantile(0.50, container_memory_usage_bytes) by (namespace)
```

Step 2: 95th Percentile

Calculate the 95th percentile memory usage per namespace:

```
quantile(0.95, container_memory_usage_bytes) by (namespace)
```

Compare this with the maximum values. What does this tell you about the distribution?

Step 3: Standard Deviation

Calculate the standard deviation of memory usage per namespace:

```
stddev(container_memory_usage_bytes) by (namespace)
```

Which namespace has the most variable memory usage?

Exercise 4: Top-K and Bottom-K

Practice identifying the top and bottom resource consumers.

Step 1: Top 5 Memory Consumers

Find the top 5 containers by memory usage:

```
topk(5, container_memory_usage_bytes)
```

Note which containers are using the most memory.

Step 2: Top 3 Namespaces by Total Memory

Find the top 3 namespaces by total memory usage:

```
topk(3, sum(container_memory_usage_bytes) by (namespace))
```

Step 3: Bottom 5 CPU Consumers

Find the bottom 5 containers by CPU usage:

```
bottomk(5, container_cpu_usage_seconds_total)
```

Exercise 5: Combining Aggregations

Practice combining multiple aggregation functions and filters.

Step 1: Average of Maximums

Calculate the average of the maximum memory usage per pod, grouped by namespace:

```
avg(max(container_memory_usage_bytes) by (namespace, pod)) by (namespace)
```

This shows the average peak memory usage per pod in each namespace.

Step 2: Filtered Aggregation

Calculate the total memory usage only for namespaces that match a specific pattern (e.g., starting with "app-"):

```
sum(container_memory_usage_bytes{namespace=~"^app-.*"}) by (namespace)
```

Adjust the pattern to match namespaces in your cluster.

Step 3: Multi-Label Grouping

Group memory usage by both namespace and node:

```
sum(container_memory_usage_bytes) by (namespace, node)
```

This shows how memory is distributed across nodes within each namespace.

Exercise 6: OpenShift-Specific Aggregations

Practice writing aggregation queries specific to OpenShift metrics.

Step 1: Pod Count per Namespace

Count the number of pods in each namespace:

```
count(kube_pod_info) by (namespace)
```

Step 2: Total CPU per Node

Calculate the total CPU usage per node:

```
sum(container_cpu_usage_seconds_total) by (node)
```

Which node has the highest CPU usage?

Step 3: Average Memory per Node

Calculate the average memory usage per node:

```
avg(container_memory_usage_bytes) by (node)
```

Compare this with the total CPU per node. Are the nodes balanced?

Verification

After completing these exercises, verify your understanding:

- Can you calculate sums, averages, and counts across time series?
- Can you use min, max, and quantile functions?
- Can you identify top and bottom resource consumers?
- Can you combine multiple aggregation functions?
- Can you group metrics by multiple labels?

If you can answer yes to all these questions, you are ready to proceed to the next module on advanced query patterns.

Module 4: Advanced Query Patterns

Content

This module covers advanced PromQL patterns including rate functions, subqueries, logical operators, and time-based functions. You will learn how to write complex queries for real-world monitoring scenarios.

Rate Functions

Rate functions calculate the per-second average rate of increase of a time series. They are essential for working with counter metrics.

The `rate()` Function

The `rate()` function calculates the per-second average rate of increase:

```
rate(http_requests_total[5m])
```

This calculates the average per-second rate of HTTP requests over the last 5 minutes.

The `irate()` Function

The `irate()` function calculates the per-second instant rate based on the last two data points:

```
irate(http_requests_total[5m])
```

Use `irate()` when you need the most recent rate and `rate()` for smoother averages.

The `increase()` Function

The `increase()` function calculates the total increase over a time range:

```
increase(http_requests_total[1h])
```

This shows the total number of requests in the last hour.

Rate with Aggregation

Combine rate functions with aggregations:

```
# Total request rate per service
sum(rate(http_requests_total[5m])) by (service)

# Average request rate per namespace
```

```
avg(rate(http_requests_total[5m])) by (namespace)
```

Working with Gauges

Gauges represent values that can go up or down. Different functions are used for gauges compared to counters.

Instant Values

For gauges, you typically query the current value:

```
container_memory_usage_bytes
```

Average Over Time

Calculate the average value over a time range:

```
avg_over_time(container_memory_usage_bytes[5m])
```

Max and Min Over Time

```
# Maximum memory usage over the last hour
max_over_time(container_memory_usage_bytes[1h])

# Minimum memory usage over the last hour
min_over_time(container_memory_usage_bytes[1h])
```

Rate of Change for Gauges

Use `delta()` or `deriv()` for gauges:

```
# Change in memory usage over 5 minutes
delta(container_memory_usage_bytes[5m])

# Per-second derivative
deriv(container_memory_usage_bytes[5m])
```

Subqueries

Subqueries allow you to evaluate an instant query at regular intervals over a time range.

Basic Subquery Syntax

```
<instant_query>[<range>:<resolution>]
```

Subquery Example

```
# Average rate over 5 minutes, evaluated every 30 seconds for the last hour
avg_over_time(rate(http_requests_total[5m])[1h:30s])
```

Common Subquery Patterns

```
# Maximum 5-minute rate over the last hour
max_over_time(rate(http_requests_total[5m])[1h:5m])

# 95th percentile of memory usage over the last day
quantile_over_time(0.95, container_memory_usage_bytes[1d:1h])
```

Logical Operators

Logical operators allow you to combine multiple conditions in queries.

AND Operator

The `and` operator returns the left-hand side if the right-hand side has a matching time series:

```
# Memory usage only for containers that also have CPU metrics
container_memory_usage_bytes and container_cpu_usage_seconds_total
```

OR Operator

The `or` operator returns all time series from both sides:

```
# Combine metrics from two different sources
metric_a or metric_b
```

Unless Operator

The `unless` operator returns the left-hand side if the right-hand side has no matching time series:

```
# Memory usage for containers without CPU metrics
container_memory_usage_bytes unless container_cpu_usage_seconds_total
```

Time-Based Functions

PromQL provides functions for working with timestamps and time ranges.

`time()` Function

Returns the current time in seconds since the Unix epoch:

```
time()
```

`timestamp()` Function

Returns the timestamp of each sample:

```
timestamp(container_memory_usage_bytes)
```

Time Calculations

Calculate time differences or offsets:

```
# Time 1 hour ago
time() - 3600

# Metrics from 1 hour ago (if available)
container_memory_usage_bytes offset 1h
```

Mathematical Functions

PromQL includes various mathematical functions for transforming values.

Basic Math Functions

```
# Absolute value
abs(container_memory_usage_bytes - 1073741824)

# Square root
sqrt(container_memory_usage_bytes)

# Logarithm (natural log)
ln(container_memory_usage_bytes)

# Logarithm base 10
log10(container_memory_usage_bytes)

# Exponentiation
exp(rate(http_requests_total[5m]))
```

Rounding Functions

```
# Round to nearest integer
round(container_memory_usage_bytes / 1048576)

# Round up
ceil(container_memory_usage_bytes / 1048576)

# Round down
floor(container_memory_usage_bytes / 1048576)
```

Label Manipulation

Functions for modifying labels in query results.

`label_replace()`

Add or modify labels:

```
label_replace(
  container_memory_usage_bytes,
  "environment",
  "$1",
  "namespace",
  "(.*)-prod"
)
```

`label_join()`

Join multiple label values:

```
label_join(
  container_memory_usage_bytes,
  "full_name",
  "-",
  "namespace",
  "pod",
  "container"
)
```

Advanced Patterns for OpenShift

CPU Utilization Percentage

Calculate CPU utilization as a percentage:

```
# CPU utilization per container
100 * rate(container_cpu_usage_seconds_total[5m])
```

```
# Average CPU utilization per namespace
avg(100 * rate(container_cpu_usage_seconds_total[5m])) by (namespace)
```

Memory Utilization Percentage

Calculate memory utilization relative to limits:

```
# Memory utilization percentage (if limits are available)
100 * container_memory_usage_bytes / container_spec_memory_limit_bytes
```

Request Rate with Error Rate

Calculate error rates:

```
# Error rate as percentage of total requests
100 * sum(rate(http_requests_total{status=~"5.."}[5m])) by (service) /
  sum(rate(http_requests_total[5m])) by (service)
```

Pod Restart Rate

Monitor pod restarts:

```
# Rate of pod container restarts
rate(kube_pod_container_status_restarts_total[5m])
```

Resource Quota Usage

Monitor resource quota usage:

```
# Memory quota usage percentage
100 * sum(container_memory_usage_bytes) by (namespace) /
  kube_resourcequota{resource="requests.memory", type="used"}
```

Query Optimization

Avoid Overly Long Ranges

Long ranges increase query time and memory usage:

```
# Good: 5-minute range
rate(http_requests_total[5m])

# Less efficient: 1-day range
rate(http_requests_total[1d])
```

Use Appropriate Resolution

For subqueries, use appropriate resolution:

```
# Good: 1-minute resolution
avg_over_time(rate(http_requests_total[5m])[1h:1m])

# Less efficient: 1-second resolution
avg_over_time(rate(http_requests_total[5m])[1h:1s])
```

Limit Result Size

Use `topk()` or `bottomk()` to limit results:

```
# Only top 10 results
topk(10, container_memory_usage_bytes)
```

Debugging Queries

Check for Missing Metrics

Use `absent()` to detect if a metric is missing:

```
# Returns 1 if metric is missing, empty if present
absent(http_requests_total)
```

Validate Label Values

Check what label values exist:

```
# See all namespace values
group by (namespace) (container_memory_usage_bytes)
```

Test Query Components

Break complex queries into parts:

```
# Test the inner query first
rate(http_requests_total[5m])

# Then add aggregation
sum(rate(http_requests_total[5m])) by (service)
```

Summary

In this module, you learned:

- Rate functions: `rate()`, `irate()`, and `increase()` for counters
- Functions for gauges: `avg_over_time()`, `max_over_time()`, `delta()`, `deriv()`
- Subqueries for time-based analysis
- Logical operators: `and`, `or`, `unless`
- Time-based functions: `time()`, `timestamp()`, `offset`
- Mathematical functions for value transformation
- Label manipulation functions
- Advanced patterns for OpenShift monitoring
- Query optimization techniques
- Debugging strategies for complex queries

You now have the knowledge to write sophisticated Prometheus queries for real-world monitoring scenarios. Practice these patterns in your OpenShift environment to become proficient with PromQL.

Exercises

Exercise 1: Rate Functions

Practice using rate functions to calculate per-second rates from counter metrics.

Step 1: Basic Rate Calculation

If you have HTTP request metrics, calculate the request rate:

```
rate(http_requests_total[5m])
```

If you don't have HTTP metrics, use a counter metric available in your cluster. You can find available metrics using:

```
curl -H "Authorization: Bearer $(oc whoami -t)" \
  "https://prometheus-k8s-openshift-monitoring.apps.<cluster-
domain>/api/v1/label/__name__/values"
```

Step 2: Rate with Aggregation

Calculate the total request rate per service:

```
sum(rate(http_requests_total[5m])) by (service)
```

Step 3: Compare rate() and irate()

Compare the results of `rate()` and `irate()`:

```
# Average rate
rate(http_requests_total[5m])

# Instant rate
irate(http_requests_total[5m])
```

Observe the differences. When would you use each?

Exercise 2: Gauge Functions

Practice using functions designed for gauge metrics.

Step 1: Average Over Time

Calculate the average memory usage over the last 5 minutes:

```
avg_over_time(container_memory_usage_bytes[5m])
```

Step 2: Maximum Over Time

Find the maximum memory usage over the last hour:

```
max_over_time(container_memory_usage_bytes[1h])
```

Compare this with the current value. What does this tell you?

Step 3: Delta Function

Calculate the change in memory usage over 5 minutes:

```
delta(container_memory_usage_bytes[5m])
```

Positive values indicate memory increase, negative values indicate decrease.

Exercise 3: Subqueries

Practice writing subqueries for time-based analysis.

Step 1: Basic Subquery

Calculate the average 5-minute rate over the last hour, evaluated every minute:

```
avg_over_time(rate(http_requests_total[5m])[1h:1m])
```

If you don't have HTTP metrics, adapt this to a counter metric in your cluster.

Step 2: Maximum Rate Over Time

Find the maximum 5-minute rate over the last 6 hours:

```
max_over_time(rate(http_requests_total[5m])[6h:5m])
```

Step 3: Percentile Over Time

Calculate the 95th percentile of memory usage over the last day:

```
quantile_over_time(0.95, container_memory_usage_bytes[1d:1h])
```

Exercise 4: Logical Operators

Practice using logical operators to combine conditions.

Step 1: AND Operator

Find containers that have both memory and CPU metrics:

```
container_memory_usage_bytes and container_cpu_usage_seconds_total
```

How many containers match this condition?

Step 2: UNLESS Operator

Find containers with memory metrics but no CPU metrics:

```
container_memory_usage_bytes unless container_cpu_usage_seconds_total
```

Step 3: Complex Logical Expression

Combine multiple conditions. For example, find memory usage for containers in production namespaces that also have CPU metrics:

```
container_memory_usage_bytes{namespace=~"prod.*"} and
```

```
container_cpu_usage_seconds_total
```

Exercise 5: Mathematical Functions

Practice using mathematical functions to transform metric values.

Step 1: Convert Bytes to Megabytes

Convert memory usage from bytes to megabytes:

```
container_memory_usage_bytes / 1048576
```

Step 2: Round Values

Round memory usage in megabytes to the nearest integer:

```
round(container_memory_usage_bytes / 1048576)
```

Step 3: Calculate Percentage

If you have limit metrics, calculate memory usage as a percentage:

```
100 * container_memory_usage_bytes / container_spec_memory_limit_bytes
```

Exercise 6: Advanced OpenShift Patterns

Practice writing advanced queries for OpenShift-specific monitoring scenarios.

Step 1: CPU Utilization Percentage

Calculate CPU utilization as a percentage per namespace:

```
sum(100 * rate(container_cpu_usage_seconds_total[5m])) by (namespace)
```

Which namespace has the highest CPU utilization?

Step 2: Pod Restart Rate

Calculate the rate of pod container restarts:

```
sum(rate(kube_pod_container_status_restarts_total[5m])) by (namespace, pod)
```

Are there any pods with high restart rates?

Step 3: Top Memory Consumers with Rate

Find the top 5 containers by memory usage, showing their current values:

```
topk(5, container_memory_usage_bytes)
```

Now find the top 5 by average memory usage over the last hour:

```
topk(5, avg_over_time(container_memory_usage_bytes[1h]))
```

Compare the results. Are they the same?

Exercise 7: Query Optimization

Practice optimizing queries for better performance.

Step 1: Compare Range Durations

Compare query execution time for different range durations:

```
# Short range
rate(http_requests_total[1m])

# Longer range
rate(http_requests_total[15m])
```

Note the execution time difference (if visible in your Prometheus UI).

Step 2: Limit Results

Instead of returning all results, limit to top 10:

```
# All results
container_memory_usage_bytes

# Top 10 only
topk(10, container_memory_usage_bytes)
```

Step 3: Optimize Subquery Resolution

Compare subqueries with different resolutions:

```
# Higher resolution (more data points)
avg_over_time(rate(http_requests_total[5m])[1h:30s])

# Lower resolution (fewer data points)
```

```
avg_over_time(rate(http_requests_total[5m])[1h:5m])
```

Use the resolution that provides sufficient detail without excessive computation.

Exercise 8: Debugging Queries

Practice debugging techniques for complex queries.

Step 1: Check for Missing Metrics

Check if a specific metric exists:

```
absent(nonexistent_metric)
```

This should return 1 if the metric is missing. Try with an existing metric to see it return empty.

Step 2: Break Down Complex Queries

Start with a simple query and build up complexity:

```
# Step 1: Basic metric
container_memory_usage_bytes

# Step 2: Add filter
container_memory_usage_bytes{namespace="default"}

# Step 3: Add aggregation
sum(container_memory_usage_bytes{namespace="default"})

# Step 4: Add rate (if applicable)
sum(rate(container_memory_usage_bytes{namespace="default"}[5m]))
```

Step 3: Validate Label Values

Check what label values exist for a specific label:

```
# See all unique namespace values
count by (namespace) (container_memory_usage_bytes)
```

Verification

After completing these exercises, verify your understanding:

- Can you calculate rates from counter metrics?
- Can you use functions appropriate for gauge metrics?
- Can you write subqueries for time-based analysis?

- Can you use logical operators to combine conditions?
- Can you apply mathematical functions to transform values?
- Can you write optimized queries for OpenShift monitoring?
- Can you debug complex queries effectively?

If you can answer yes to all these questions, you have successfully completed the Prometheus Query Workshop! You are now equipped to write effective PromQL queries for monitoring your OpenShift 4.16 infrastructure.